Campo	Detalle
Qué le pregunté a la IA	¿Qué exige la interfaz Vendible y por qué Catalogo::agregar(Vendible $articulo) rechaza un new DateTime()?
Qué me respondió	Vendible exige codigo(), nombre(), precioVenta(int), estaDisponible() y vender(int). Cualquier clase que la implemente firma un contrato. El *type hinting* del parámetro obliga a que el argumento implemente la interfaz; un DateTime no lo hace, así que PHP lanza TypeError antes de ejecutar el método.
¿Lo usé? ¿Qué corregí y por qué?	Sí. Verifiqué que Articulo implements Vendible y que Catalogo::agregar recibiera Vendible. Probé Catalogo::agregar(new DateTime()) y confirmé el TypeError.
Cómo lo verifiqué	Ejecuté en la consola: TypeError: Catalogo::agregar(): Argument #1 ($articulo) must be of type Apicultores\Contratos\Vendible, DateTime given. La prueba 6 de php verificar.php pasó a ☑.

Interacción 6 — Polimorfismo: mismo método, distinto resultado

Campo	Detalle
Qué le pregunté a la IA	¿Por qué 12 unidades de miel cuestan 486 pero 12 personas en el taller cuestan 2400, si ambos llaman a precioVenta(12)?
Qué me respondió	Porque PHP resuelve el método en tiempo de ejecución según el tipo real del objeto (polimorfismo). Producto::precioVenta aplica el 10 % desde 12 unidades; Servicio::precioVenta no aplica descuento porque se cobra por persona o por día. El código cliente no sabe qué tipo recibe: solo llama precioVenta(12) y PHP decide.
¿Lo usé? ¿Qué corregí y por qué?	Sí. Completé Servicio::estaDisponible() (TODO 4) con return true; y Catalogo::servicios() (TODO 5) con array_values(array_filter($this->articulos, fn(Vendible $a): bool => $a instanceof Servicio));
Cómo lo verifiqué	Ejecuté php demo.php y la línea 4 mostró "12 taller → Bs 2.400,00". La línea 8 mostró "12 MIE-500 + 1 SRV-EXT → Bs 636,00". La prueba 9 de php verificar.php pasó a ☑.

Interacción 7 — Trait frente a herencia

Campo	Detalle
Qué le pregunté a la IA	¿En qué se diferencia un trait de una clase padre y por qué enBolivianos() es protected?
Qué me respondió	Un trait no es herencia: es código que se "inyecta" horizontalmente en varias clases sin importar su jerarquía, y una clase puede usar varios traits. Una clase padre define "qué es" el objeto y permite instanceof. enBolivianos() es protected porque es un detalle interno de la clase; el exterior usa precioFormateado(), que es public. Eso es encapsulamiento.
¿Lo usé? ¿Qué corregí y por qué?	Sí. Confirmé que Articulo use ConBolivianos y que enBolivianos() sea protected. Llamé desde precioFormateado(): return $this->enBolivianos($this->precio);
Cómo lo verifiqué	Ejecuté php demo.php y la línea 6 mostró "Trait: Apicultores\Soporte\ConBolivianos → Bs 45,00". La prueba 10 de php verificar.php pasó a ☑.

Verificación final

php verificar.php **— Resultado: 14 de 14 comprobaciones** ☑

#	Comprobación	Estado
1	composer.json declara PSR-4: Apicultores\ → src/	☑
2	El autoload encuentra clases, interfaz, trait y enum por su espacio de nombres	☑
3	Las páginas no usan require de src/: solo vendor/autoload.php	☑
4	Enum Categoria: from, tryFrom, cases y descripcion()	☑
5	Articulo es abstracta: no se puede instanciar	☑
6	Producto y Servicio heredan de Articulo e implementan Vendible y JsonSerializable	☑
7	Polimorfismo: 2 miel = 90 · 12 miel = 486 (−10 %) · 12 personas al taller = 2400 (sin descuento)	☑
8	Producto no vende sin stock; un servicio siempre está disponible	☑

9	Trait ConBolivianos: Articulo lo usa y su método es protected	☑
10	Catálogo: 6 productos + 2 servicios, 7 disponibles, inventario 5030 (sin servicios)	☑
11	totalPedido: 12 MIE-500 + 1 SRV-EXT = 636; código inexistente → DomainException	☑
12	json_encode(Producto) usa jsonSerialize con parent:: (tipo, código, categoría, stock)	☑
13	La página muestra categorías del enum, precio por docena y servicios	☑
14	api.php responde JSON con los 8 artículos	☑

Plataforma y entorno

Elemento	Detalle
Versión de PHP	8.5.11 (cli)
Extensiones activadas	curl, mbstring
Archivo de configuración	C:\php\php.ini
Servidor embebido	php -S 127.0.0.1:8000 -t public
Ruta de ejecución	C:\Laboratorio 6\TW2\lab06\inicio
Composer	2.x (composer -V)

Archivos modificados y su función

- **src/Modelo/Categoria.php (1 TODO resuelto)**

#	Función / Regla	Técnica implementada
1	Describir cada caso del enum	match ($this) → Miel, Derivados, Cera

- **src/Modelo/Producto.php (2 TODO resueltos)**

#	Función / Regla	Técnica implementada
1	Precio mayorista	−10 % desde 12 unidades → round($subtotal − $subtotal * 0.10, 2)
2	JSON completo	parent::jsonSerialize() + ['categoria', 'stock', 'socio']

- **src/Modelo/Servicio.php (1 TODO resuelto)**

#	Función / Regla	Técnica implementada
1	Disponibilidad de servicio	return true; (sin stock, siempre disponible)

- **src/Catalogo.php (1 TODO resuelto)**

#	Función / Regla	Técnica implementada
1	Filtrar servicios	array_filter con fn(Vendible $a) => $a instanceof Servicio

- **public/api.php (1 TODO resuelto)**

#	Función / Regla	Técnica implementada
1	Autoload	require __DIR__ . '/../vendor/autoload.php' (reemplaza los require manuales)

- **Verificador.php (observación, sin modificar)**

#	Función / Regla	Técnica observada
1	Warning de PHP 8.5	curl_close() está deprecado desde 8.5; no afecta el resultado (14 de 14)

TÍTULO 2 — Reflexión

En el Laboratorio 06 aprendí a estructurar un proyecto PHP profesional usando Composer y PSR-4. A diferencia del Laboratorio 05, donde el código se cargaba con require manuales, ahora vendor/autoload.php resuelve todo automáticamente: el namespace Apicultores\ mapea a src/ y basta con declarar el use para que la clase correcta se cargue. Reemplacé los require de api.php por una sola línea de autoload, y entendí por qué con 40 clases el require manual sería insostenible.

Comprendí la diferencia entre clase abstracta, interfaz y trait. Articulo es abstracta porque representa un concepto genérico ("algo vendible") y obliga a las hijas a implementar tipo() y detalle(). Vendible es la interfaz que define el contrato: codigo(), nombre(), precioVenta(), estaDisponible() y vender(). ConBolivianos es un trait: código reutilizable que se inyecta horizontalmente sin romper la jerarquía. Ahora sé cuándo usar cada uno.

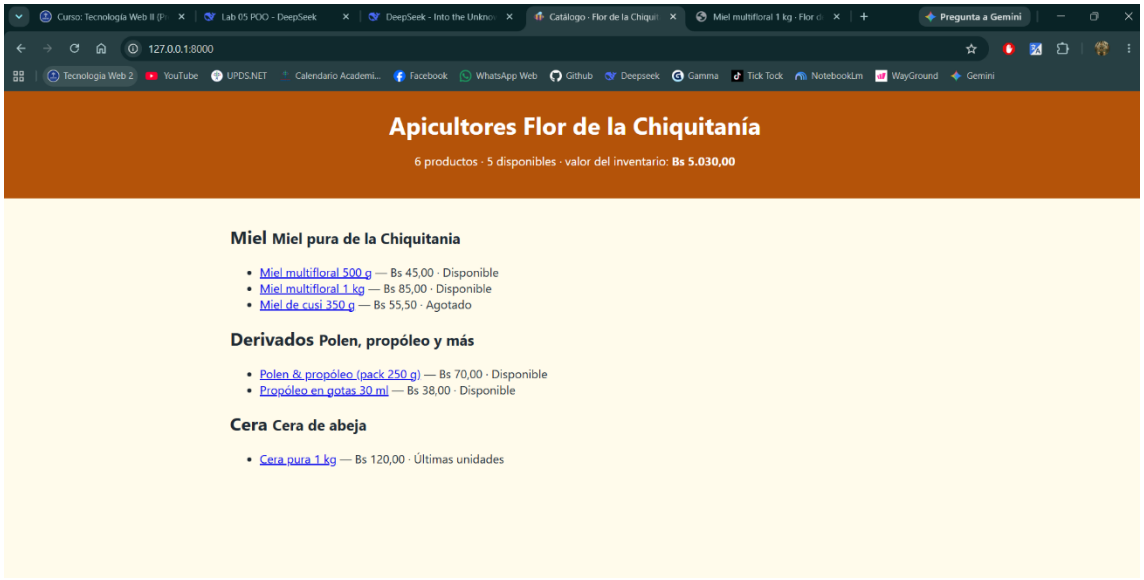
Reforcé el polimorfismo real: Catalogo::totalPedido() no sabe si recibe un Producto o un Servicio; solo llama precioVenta() y PHP decide en tiempo de ejecución qué implementación usar. Por eso 12 miel cuestan 486 (con −10 %) y 12 personas en el taller cuestan 2400 (sin descuento). El *type hinting* Vendible $articulo impide que se cuele un DateTime con un TypeError inmediato.

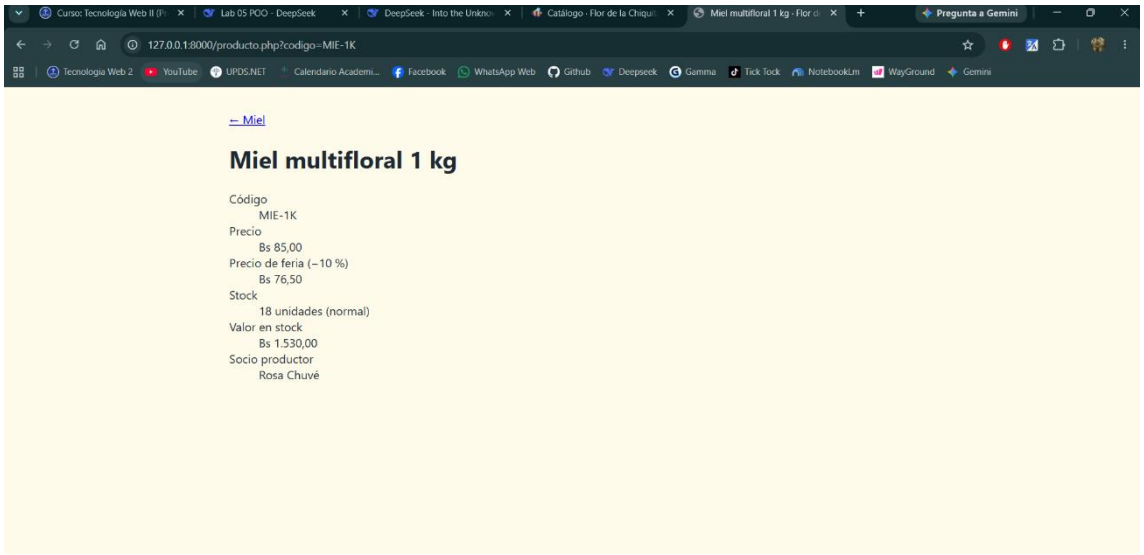
Aprendí también a completar jsonSerialize() con parent:: para reutilizar lo del padre y agregar solo lo propio. El operador + en arrays preserva las claves, por eso Producto puede añadir categoria, stock y socio sin perder tipo, codigo, nombre y precio. Antes del TODO 3, el JSON de /api.php estaba incompleto.

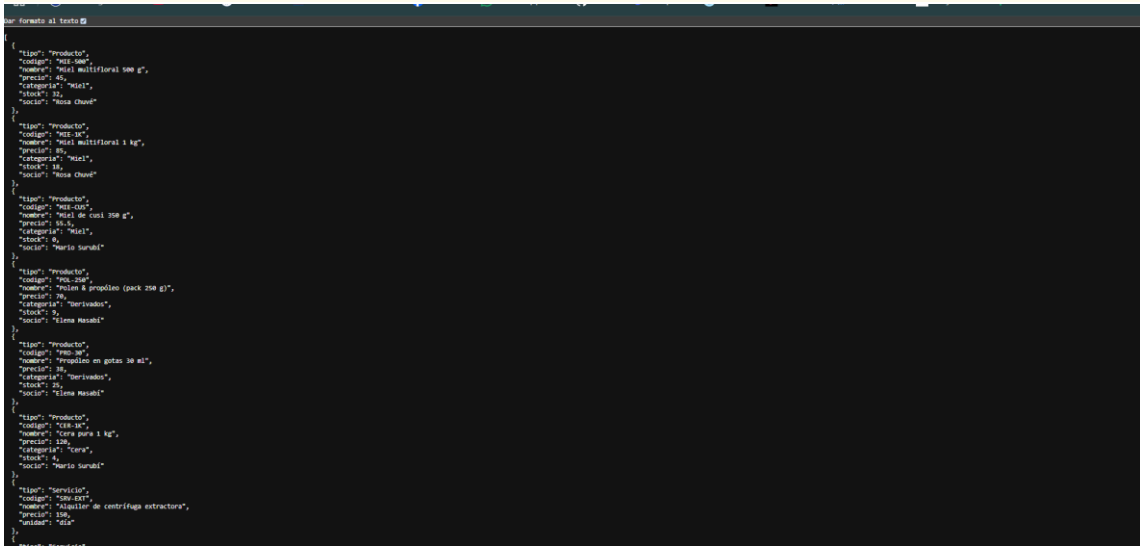
Finalmente, entendí por qué Servicio::estaDisponible() devuelve true: un servicio no tiene stock, así que alquileres y talleres están siempre disponibles. Y por qué Catalogo::servicios() filtra con instanceof Servicio: el catálogo es polimórfico y no debe confundir un producto con un servicio.

Al terminar el laboratorio, el catálogo cumple con la consigna: 6 productos + 2 servicios, 7 disponibles, inventario de Bs 5.030,00, totalPedido de 636 para 12 miel + 1 centrífuga, y JSON de 8 artículos en /api.php. El sistema es extensible: si mañana agrego una Canasta, solo debe implementar Vendible y Catalogo la acepta sin cambios.

Capturas:



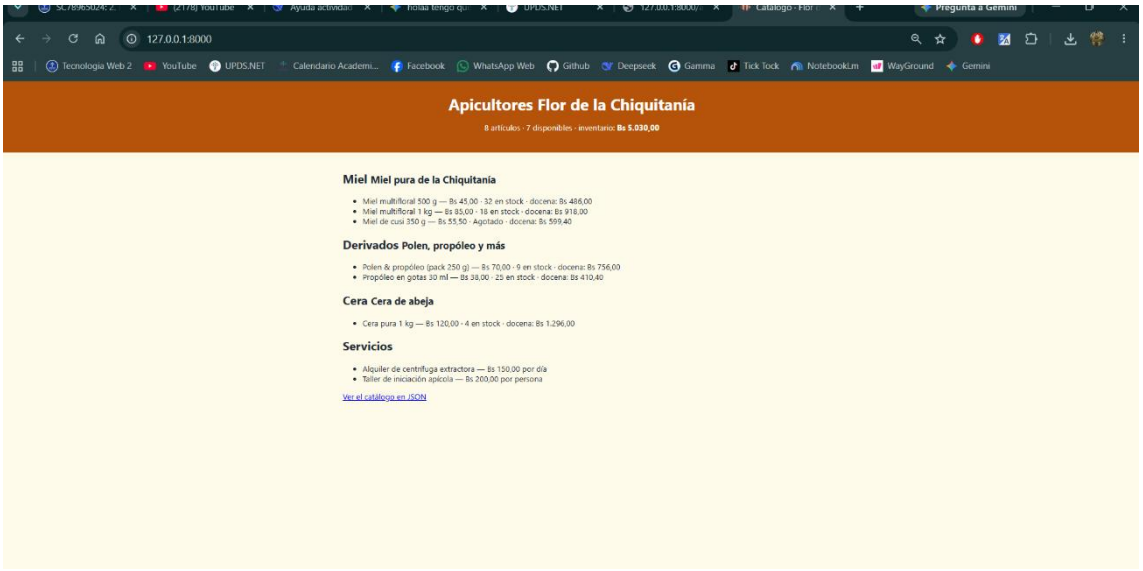





```
PS C:\Laboratorio 6\TW2\lab06\inicio> php verificar.php
[OK] composer.json declara PSR-4: Apicultores\ → src/
[OK] El autoload encuentra clases, interfaz, trait y enum por su espacio de nombres
[OK] Las páginas no usan require de src/: solo vendor/autoload.php
[OK] Enum Categoria: from, tryFrom, cases y descripcion()
[OK] Articulo es abstracta: no se puede instanciar
[OK] Producto y Servicio heredan de Articulo e implementan Vendible y JsonSerializable
[OK] Polimorfismo: 2 miel = 90 · 12 miel = 486 (−10 %) · 12 personas al taller = 2400 (sin descuento)
[OK] Producto no vende sin stock; un servicio siempre está disponible
[OK] Trait ConBolivianos: Articulo lo usa y su método es protected
[OK] Catálogo: 6 productos + 2 servicios, 7 disponibles, inventario 5030 (sin servicios)
[OK] totalPedido: 12 MIE-500 + 1 SRV-EXT = 636; código inexistente → DomainException
[OK] json_encode(Producto) usa jsonSerialize con parent:: (tipo, código, categoría, stock)
PHP Deprecated:  Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\
TW2\lab06\inicio\Verificador.php on line 57

Deprecated: Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\TW2\l
ab06\inicio\Verificador.php on line 57
[OK] La página muestra categorías del enum, precio por docena y servicios
PHP Deprecated:  Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\
TW2\lab06\inicio\Verificador.php on line 57

Deprecated: Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\TW2\l
ab06\inicio\Verificador.php on line 57
[OK] api.php responde JSON con los 8 artículos
Resultado: 14 de 14 comprobaciones
```


127.0.0.1:8000
Tecnologia Web 2
YouTube
UPDS.NET
Calendario Academi...
Facebook
WhatsApp Web
Github
Deepseek
Gamma
Tick Tock
NotebookLm
WayGround
Gemini
Apicultores Flor de la Chiquitanía
8 artículos · 7 disponibles · inventario: Bs 5.030,00
Miel Miel pura de la Chiquitanía
Miel multifloral 500 g — Bs 45,00 · 32 en stock · docena: Bs 486,00
Miel multifloral 1 kg — Bs 85,00 · 18 en stock · docena: Bs 918,00
Miel de cusi 350 g — Bs 55,50 · Agotado · docena: Bs 599,40
Derivados Polen, propóleo y más
Polen & propóleo (pack 250 g) — Bs 70,00 · 9 en stock · docena: Bs 756,00
Propóleo en gotas 30 ml — Bs 38,00 · 25 en stock · docena: Bs 410,40
Cera Cera de abeja
Cera pura 1 kg — Bs 120,00 · 4 en stock · docena: Bs 1.296,00
Servicios
Alquiler de centrífuga extractora — Bs 150,00 por día
Taller de iniciación apícola — Bs 200,00 por persona
Ver el catálogo en JSON